

UNIVERSIDAD CENTRAL DEL ECUADOR
FACULTAD DE INGENIERIA Y CIENCIAS APLICADAS
PROGRAMACION AVANZADA I

NOMBRE: Gabriel Piña

CubeFaceColor

Primeramente se observa y se trabaja con un private final float[] cubeCoords que serian las coordenadas del cubo que trabaja con los 24 vértices y se distribuye 4 por cada cara. Y se divide en frontal, atrás, izquierda, derecha, superior e inferior.

Luego tenemos private final float[] colors que es donde especificamos los colores en este caso el color de cada cara. Podemos agregar el color para cada cara como en la frontal en este caso Rojo, atrás de Verde, Izquierda de Azul, Derecha de Amarillo, Superior de Magenta e Inferior de Cian. Igualmente se trabaja con el formato de colores RGBA.

Vemos igual VertexShaderCode donde ya vemos el attribute aColor y el varying Vcolor. Ahí ya decimos vColor es igual a aColor.

CubeGradient

Igualmente tenemos un private final float[] cubeCoords con las coordenadas de cada vertice en total 24, 4 por cada cara.

Luego tenemos private final float[] colors ahí donde ocurre la magia ya que se colocan los colores para generar el desgradado. Se observa que se coloca varios colores en una misma cara. En la cara frontal tenemos 4 colores que serian rojo, amarillo, lima y azul. En la cara de atrás tenemos fucsia, aqua, blanco y negro. En la izquierda tenemos el rojo, lima y azul. En el derecho tenemos fucsia, aqua y amarillo. En la cara superior tenemos el rojo, lima, azul y amarillo y en la cara inferior tenemos fucsia, aqua, blanco y negro. Ahí es donde se mezclan esos 4 colores para generar un desgradado.

Vemos igual VertexShaderCode donde ya vemos el attribute aColor y el varying Vcolor. Ahí ya decimos vColor es igual a aColor.

CubePositionColor

Empezamos con private final float[] cubeCoords que en este cubo se trabaja con 8 vertices en la cual se une con el índice o orden de unión. Igual con 4 en cada cara. En este caso se trabaja con menos vértices por trabajo que realiza private final short[] drawOrder.

En el vertexShaderCode se observa que hay una operación que convierte las coordenadas [-1,1] a rango [0,1]. Para el color en la parte vColor se explica que el color que se tomara en todo el cubo será el blanco y la operación q se observa es que los 3 ejes de X,Y y Z tomaran el mismo valor, en este caso 1.0 que arroja el color blanco como muestra en la pantalla.

CubeVertexColor

Este cubo igualmente trabaja con `private final float[] cubeCoords` que especifica las coordenadas del cubo que serian 8 vertices. La cual forman la figura final con `private final short[] drawOrder` ahí se unen cada vértices.

Aquí los colores se trabajan algo parecido como el Gradient en cada cara se trabaja con 4 colores. Para el frontal se trabaja con el rojo, verde, azul y amarillo. Para la derecha se trabaja con el azul, amarillo, blanco y negro. Atrás con blanco, negro, magenta y cian. En la izquierda los colores magenta, cian, rojo y verde. Arriba están magenta, rojo, amarillo y negro. Abajo verde, cian, blanco y azul. Igual se trabaja con un color diferente por vértice.

cubeAnimated

`cubeCoords` se trabaja con 8 vertices en total. De igual forma se trabaja uniendo cada cara con `private final short[] drawOrder` y así unir cada vertice para armar la cara de 4 vertices.

Se observa que en el `public void draw(float[] mvpMatrix)` se implementa como se trabajara para que el cubo se vea animado es decir que se le vea un cambio de colores.

Se pudo observar que se trabaja con un `float time` que dira el tiempo que cada color durara para cambiar al otro color configurado. Luego están los `float` de cada color RGB. Se explica en cada color el tiempo que durara mostrando. Tenemos el `Red*0.001`, el `Green*0.002` y el `Blue*0.003`. Con eso podemos observar el cambio de color y transición al otro.